# Abstract

This paper documents `template_search_project`, the literature-search exemplar shipped with the Research Project Template. The project demonstrates **two configurable, reproducible pipelines** sharing the same configuration file and the same `infrastructure/search/` + `infrastructure/reference/` modules. The standard pipeline (`scripts/run_search_pipeline.py`) handles a single `SearchQuery` end-to-end. The deep-search pipeline (`scripts/run_deep_search.py`, see sec. 8) fans out across a list of keywords (each capped at 100 papers per keyword from `project_config.deep_search.max_results_per_keyword` in `manuscript/config.yaml`), fully enriches every paper with its abstract and PDF fulltext, and (optionally) uses the local LLM to write a multi-section reading note for every paper. When a deep-search

aggregate exists, the latest run covered **3** keyword(s) with  unique paper(s) after cross-keyword deduplication. Both turn a free-text topic into:

# Abstract I

1. a deduplicated, year-filtered set of papers drawn from arXiv, Crossref, optional local corpora, and (opt-in) Paperclip;
2. a Pandoc-compatible `references.bib` byte-identical in style to the canonical exemplar in `template_code_project` (file `manuscript/references.bib`);
3. cached abstracts and (optionally) extracted PDF full text, written to disk under stable per-paper identifiers; and
4. an LLM-synthesised reading report assembled from per-paper analyses and a cross-corpus thematic synthesis, all produced by a local Ollama model with pinned seed and temperature.

All discovery logic lives in `infrastructure/search/literature/` (source on GitHub); all export logic lives in `infrastructure/reference/citation/` (source on GitHub); LLM synthesis reuses the existing `infrastructure/llm/` (source on GitHub) bridge. The project itself contains only thin orchestration, manuscript prose, and a test suite — perfectly mirroring the **two-layer architecture** the template enforces.

The motivating concern is *reproducibility*: a query at time $t_0$ should produce the same results at time $t_1$ unless the cache is explicitly invalidated. This is achieved by deterministic search caching keyed on canonical query identity, on-disk caching of every fetched abstract / PDF, and pinned LLM seeds. The same `manuscript/config.yaml` that drives the pipeline is also the only configuration any reviewer needs.

**Run snapshot.** With the bundled `manuscript/config.yaml`, the most recent pipeline execution evaluated the query *"reproducible research optimization"* against local, returned 6 deduplicated paper(s) (4 carrying a DOI, 6 carrying an abstract), and recorded backend errors: none. Resolve {{…}} tokens by running `scripts/z_generate_manuscript_variables.py` after `run_search_pipeline.py`; the script writes

`output/data/manuscript_variables.json` and resolved markdown under `output/manuscript/`, which the PDF-rendering stage prefers when present.

**Claim boundary.** The committed `data/corpus.json` and the default local-source run are deterministic workflow fixtures. Their counts, summaries, and generated bibliography exercise the pipeline contract; they are not empirical findings about the literature.

**Keywords:** literature search, BibTeX automation, reproducible research, local LLM synthesis, scientific infrastructure